

Collection

Monday, 6 April 2026

9:34 AM

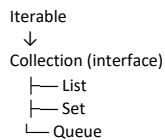
Collection in Java (Complete Notes)

What is Collection?

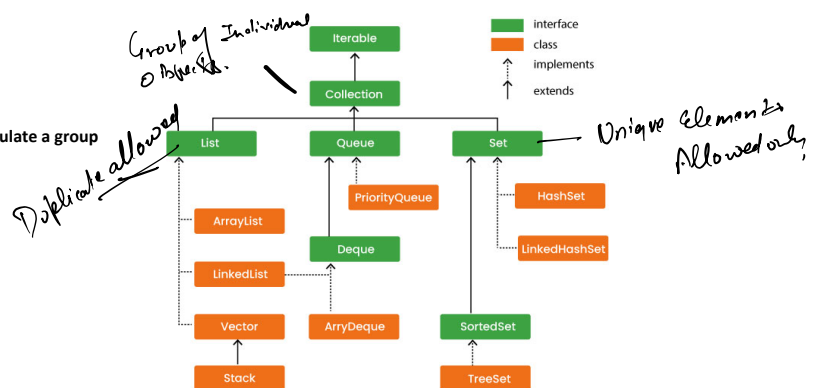
A **Collection in Java** is a framework that provides architecture to **store and manipulate a group of objects dynamically**.

- It is part of the `java.util` package
- It is based on **interfaces and classes**
- The root interface is **Collection**

Hierarchy of Collection Framework



Note: **Map** is not part of Collection interface (but part of framework)



Types of Collection

1. List (Ordered, Allows Duplicates)

- Maintains insertion order
- Allows duplicate elements

Examples: `ArrayList`, `LinkedList`, `Vector`

Example:

```
List<String> list = new ArrayList<>();  
list.add("Java");  
list.add("Python");  
list.add("Java");
```

```
System.out.println(list);
```

2. Set (No Duplicates)

- Does not allow duplicates
- Unordered (mostly)

Examples: `HashSet`, `LinkedHashSet`, `TreeSet`

Example:

```
Set<Integer> set = new HashSet<>();  
set.add(10);  
set.add(20);  
set.add(10);
```

```
System.out.println(set);
```

3. Queue (FIFO)

- First In First Out
- Used for processing tasks

Examples: `PriorityQueue`, `LinkedList`

Example:

```
Queue<Integer> q = new LinkedList<>();  
q.add(1);  
q.add(2);  
q.add(3);
```

```
System.out.println(q.poll());
```

Important Methods

Method	Work
<code>add()</code>	Insert element
<code>remove()</code>	Delete element
<code>size()</code>	Number of elements
<code>isEmpty()</code>	Check empty
<code>contains()</code>	Search element
<code>clear()</code>	Remove all

Advantages of Collection Framework

1. Dynamic Size

- Automatically resizable (no fixed size like arrays)

✓ 2. Reduces Coding Effort

- Ready-made classes like ArrayList, HashSet

✓ 3. Better Performance

- Optimized data structures for different use cases

✓ 4. Reusability

- Interfaces allow switching implementations easily

✓ 5. Type Safety (Generics)

```
List<String> list = new ArrayList<>();
```

✓ 6. Built-in Algorithms

- Sorting, searching using Collections class

✓ 7. Easy Data Manipulation

- Insert, delete, search operations are simple

✓ 8. Standard API

- Same methods across collections

✓ 9. Memory Efficient (Dynamic Allocation)

- Uses memory as needed

✓ 10. Multiple Data Structures Support

- List, Set, Queue available

▼ Disadvantages of Collection Framework

✗ 1. Performance Overhead

- Slower than arrays in simple cases

✗ 2. No Primitive Types

- Uses wrapper classes (Integer, Double)

✗ 3. More Memory Usage

- Object storage + extra overhead

✗ 4. Complexity

- Many interfaces and classes → confusing for beginners

✗ 5. Not Thread-Safe by Default

- Requires synchronization

✗ 6. Less Control

- Compared to arrays, less low-level control

✗ 7. Iteration Overhead

- Slightly slower traversal than arrays

✗ 8. Not Ideal for Real-Time Systems

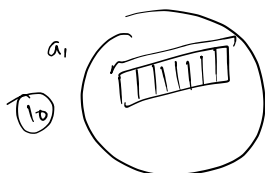
- Performance-critical systems prefer arrays

◆ Collection vs Collections

Collection	Collections
Interface	Utility class
Stores data	Provides methods (sort, reverse)

🎯 Final Conclusion (Exam Line)

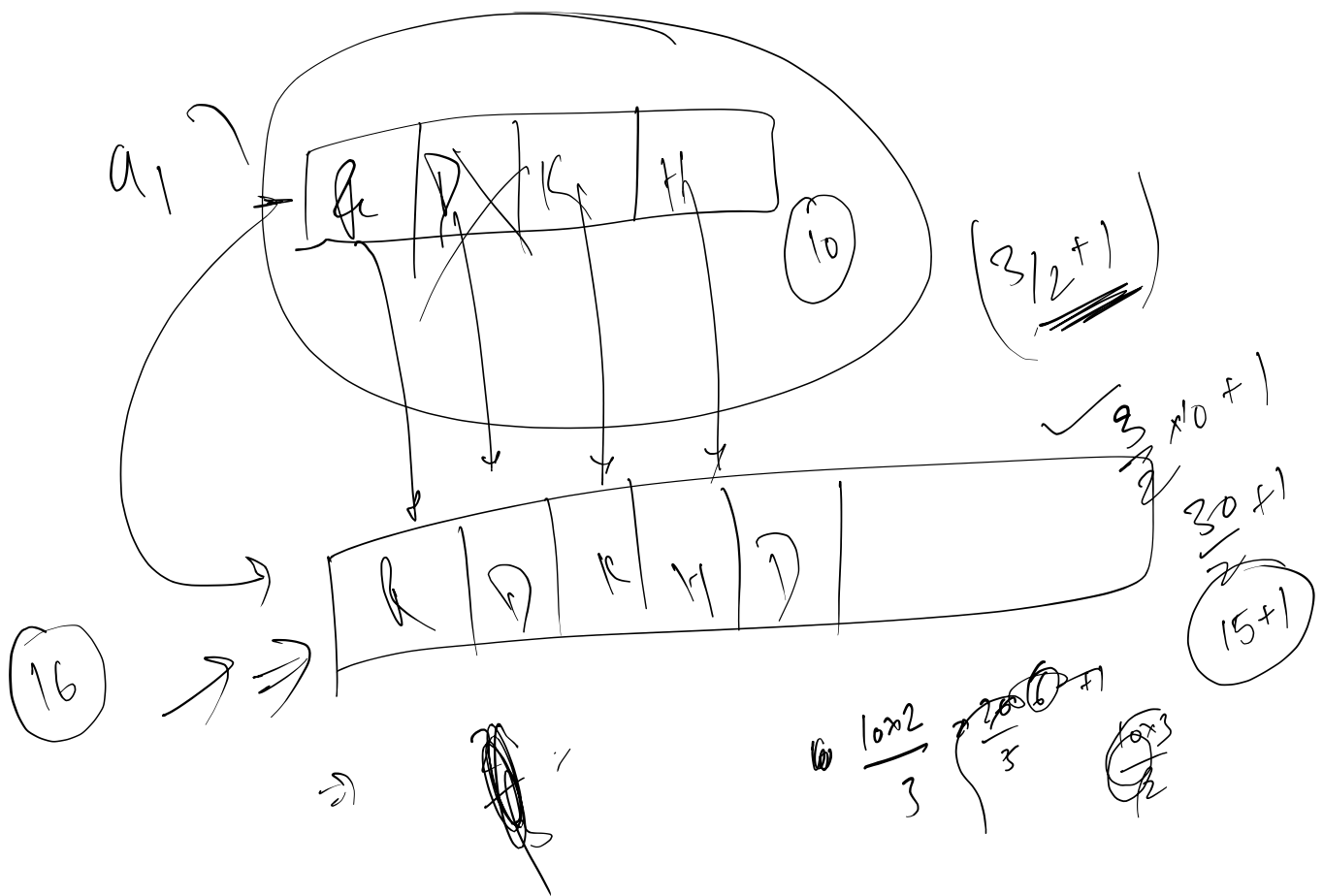
👉 "Java Collection Framework provides dynamic, reusable, and efficient data structures, but introduces overhead in memory, performance, and complexity compared to arrays."



```
public boolean add(E);  
public void add(int, E);  
public void addFirst(E);  
public void addLast(F)
```

add, addFirst, addLast

part 10: Java 8



Gemini said

Prior to Java 8, interfaces were strictly "contracts"—they could only contain abstract methods. If you added a new method to an interface, you'd break every single class that implemented it.

To solve this, Java introduced **default** and **static** methods. Here is how they work and why they matter.

1. Default Methods

Default methods allow you to add new functionality to interfaces without breaking existing implementation classes. They are marked with the `default` keyword and provide a **method body**.

- **Inheritance:** Classes implementing the interface inherit the method automatically.
- **Overriding:** Implementing classes can choose to override the default behavior or keep it as is.
- **The "Diamond Problem":** If a class implements two interfaces with the same default method signature, the compiler will throw an error. You must manually override the method in the class to resolve the conflict.

Example:

```
Java
interface Vehicle {
    void drive();
    // Default method
    default void blowHorn() {
        System.out.println("Beep beep!");
    }
}
class Car implements Vehicle {
    public void drive() {
        System.out.println("Car is moving");
    }
    // No need to implement blowHorn() unless we want custom sound
}
```

2. Static Methods

Static methods in interfaces are similar to static methods in classes. They belong to the interface itself rather than an instance of the class.

- **Utility focus:** They are perfect for providing helper methods related to the interface (e.g., validation, object creation).
- **No Inheritance:** Unlike default methods, static methods are **not inherited** by implementing classes.
- **Calling:** You must call them using the Interface name: `InterfaceName.methodName()`.

Example:

```
Java
interface MathHelper {
    static int add(int a, int b) {
        return a + b;
    }
}
// Usage:
int sum = MathHelper.add(5, 10);
```

Key Differences at a Glance

Feature	Default Method	Static Method
Keyword	default	static
Inherited?	Yes	No
Can be Overridden?	Yes	No (can be redefined in class, but not overridden)
How to call?	Via object instance: <code>obj.method()</code>	Via interface name: <code>Interface.method()</code>
Primary Purpose	Backward compatibility & optional logic.	Utility/Helper methods for the interface.

Why use them?

1. **Interface Evolution:** You can add methods to `List` or `Collection` (like `forEach` or `stream`) without forcing every developer in the world to update their custom collection classes.
2. **Code Organization:** You can keep utility methods (like `Validator.isValid()`) inside the interface they relate to, rather than creating a separate `ValidatorUtils` class.

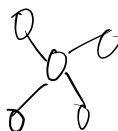
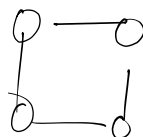
Are you running into a specific "Diamond Problem" conflict, or are you just looking to refactor some older code?

1. Syntax

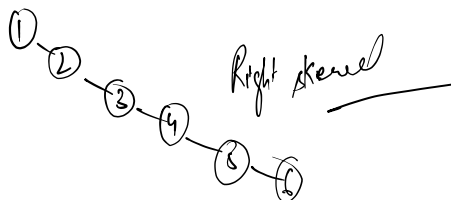
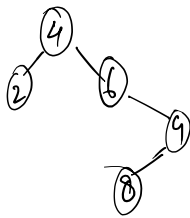
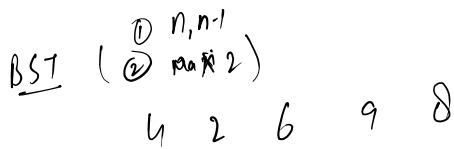
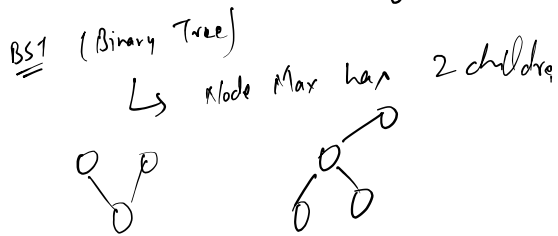
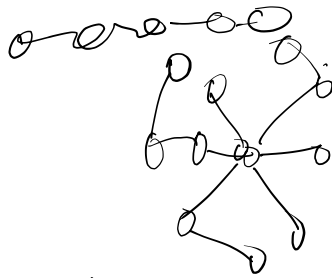
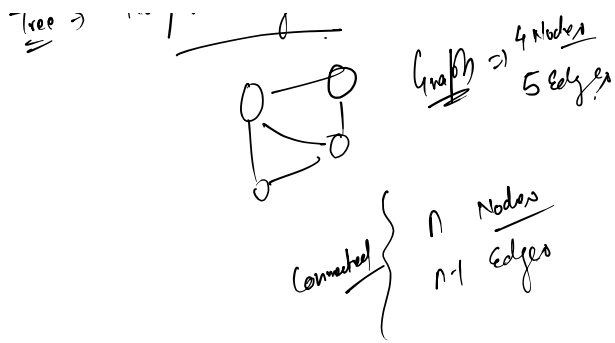
The syntax of a lambda expression consists of three parts:

1. **Parameters:** The input variables.
2. **The Arrow Token:** `->` (the "lambda operator").
3. **Body:** The expressions or statements to be executed.
 - **Zero parameters:** `() -> System.out.println("Hello");`
 - **One parameter:** `(name) -> System.out.println(name);`
 - **Multiple parameters:** `(a, b) -> a + b;`

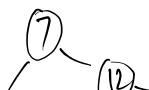
Tree

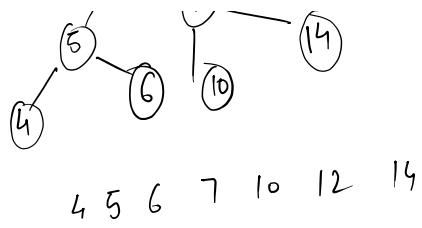


- - Model ~~data~~ Edy.co



LRR





44 32 41

44 32
01 02 (44)

01-02